Git for Developers

Module 2 — Core Git Concepts

Staging, Committing, History, and Undoing Mistakes

Module 2 — Agenda

The Three-Tree Architecture

- Working Directory, Staging Area, and Repository explained
- The Git file lifecycle: untracked → staged → committed

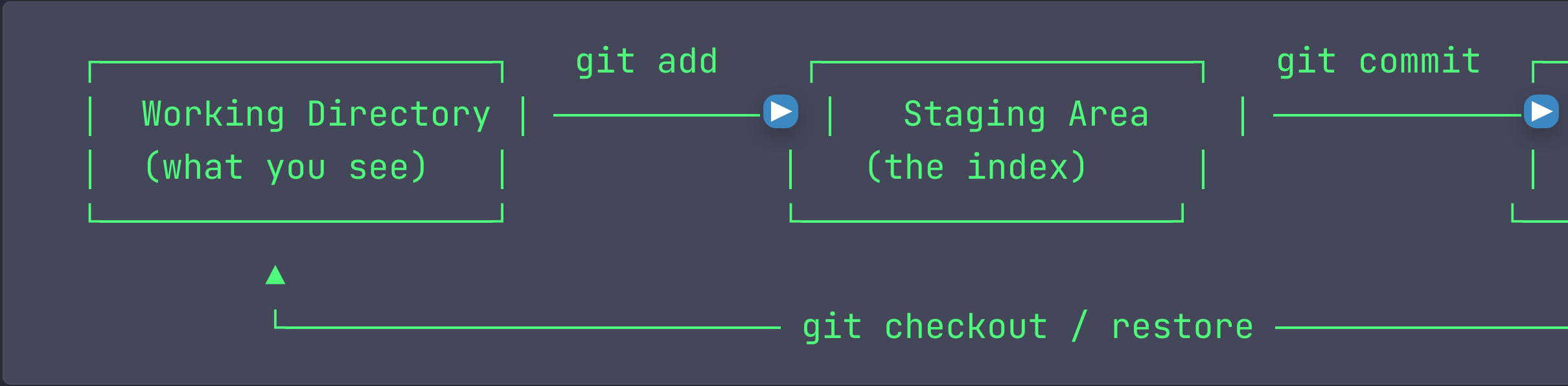
Working With Git

- Creating a repository and making your first commit
- Viewing history with `git log` and `git status`
- Undoing changes: `git restore`, `git reset`, `git revert`

Part 1 — The Three-Tree Architecture

The Three Trees

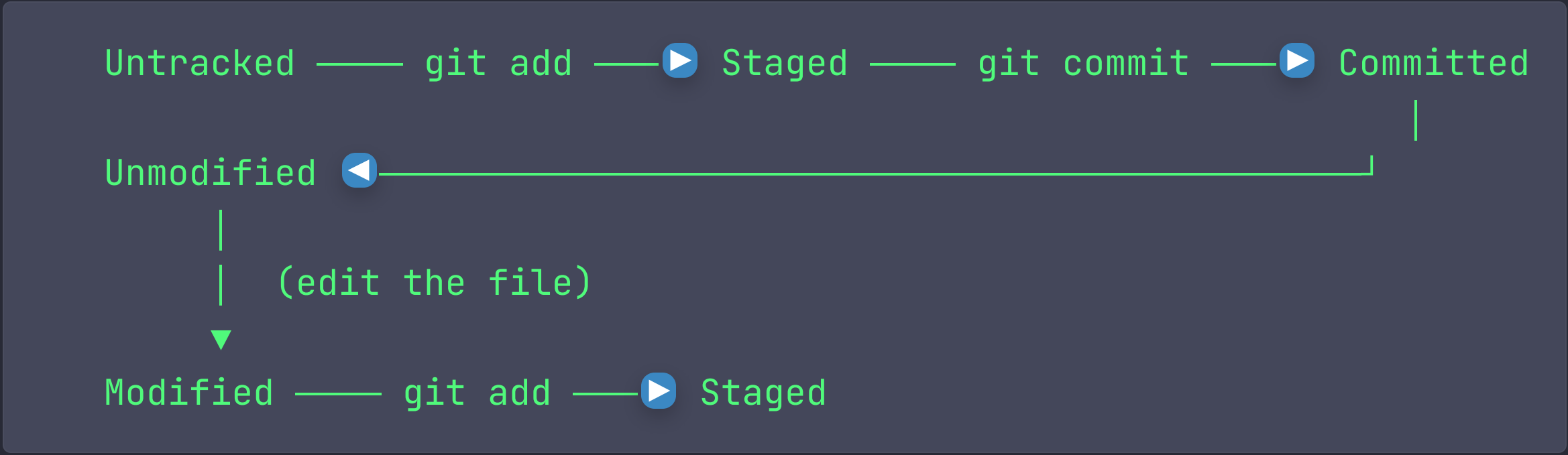
Git manages your project through three distinct areas:



Tree	Also called	Purpose
Working	Working	Your files as they exist on disk

The Git File Lifecycle

Every file in your working directory is in one of these states:



State	Meaning
Untracked	New file, Git has never seen before

Checking Status

`git status` is the most frequently used Git command:

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md          ← staged

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
        modified:   src/app.js         ← modified but not staged
```

Part 2 — Making Your First Commit

Initializing and Staging

```
# Create a new project and initialize Git
mkdir hello-git && cd hello-git
git init

# Create a file
echo "# Hello, Git!" > README.md

# Check status — file is untracked
git status

# Stage the file
git add README.md
```

Staging Strategies

```
# Stage a single file
git add README.md

# Stage multiple files
git add src/app.js src/utils.js

# Stage all changes in the current directory
git add .

# Stage parts of a file interactively (patch mode)
git add -p README.md

# Stage all tracked modified files (not new untracked files)
```

Making a Commit

```
# Commit with an inline message
git commit -m "Initial commit: add README"

# Open your configured editor to write the message
git commit

# Stage all modified tracked files and commit in one step
git commit -am "Fix typo in README"
```

What a commit records

- A SHA-1 hash (unique identifier, e.g. `a3f7c21`)

Writing Good Commit Messages

```
feat: add user authentication endpoint

Add JWT-based login and logout endpoints.
- POST /auth/login returns a signed JWT
- POST /auth/logout invalidates the token
- Includes rate limiting (5 req/min per IP)

Closes #42
```

The golden rule

- **Subject line**: 50 characters max, imperative mood ("add", "fix", "update")

Part 3 — Viewing History

`git log` — Browsing Commits

```
# Default — full log with author, date, and message
git log

# Compact one-line view
git log --oneline

# Graph view showing branches and merges
git log --oneline --graph --all

# Limit to the last N commits
git log -5

# Search commit messages
```

Reading `git log` Output

```
$ git log --oneline
a3f7c21 (HEAD → main, origin/main) feat: add user authentication
b2e1d40 fix: handle null pointer in login flow
c9a8b12 docs: update README with setup instructions
d7f3e56 chore: add .gitignore for Node.js project
e1c2a34 Initial commit: add README
```

Part	Meaning
`a3f7c21`	Abbreviated SHA hash — unique commit ID
`(HEAD → main)`	Your current position and branch

`git show` — Inspect a Commit

```
# Show the most recent commit and its diff
git show

# Show a specific commit
git show a3f7c21

# Show only the files changed in a commit
git show --stat a3f7c21

# Show a specific file at a specific commit
git show a3f7c21:src/app.js
```

`git diff` — What Changed?

```
# Changes in working directory vs. staging area (unstaged changes)
git diff

# Changes in staging area vs. last commit (staged changes)
git diff --staged

# Compare two commits
git diff a3f7c21 b2e1d40

# Compare two branches
git diff main feature/auth

# Compact summary of what changed
```

Part 4 — Undoing Mistakes

Three Ways to Undo — Overview

Scenario	Command	Safe for shared branches?
Discard unstaged changes	`git restore <file>`	Yes — local only
Unstage a file	`git restore --staged <file>`	Yes — local only
Undo the last commit (keep changes)	`git reset --soft HEAD~1`	Only if not pushed
Undo the last commit (discard changes)	`git reset --hard HEAD~1`	Only if not pushed

`git restore` — Discard Working Directory Changes

```
# Discard all changes to a file (restore to last commit)
git restore README.md

# Discard all unstaged changes in the entire repo
git restore .

# Unstage a staged file (keep the changes in working directory)
git restore --staged README.md

# Restore a file from a specific commit
git restore --source a3f7c21 README.md
```

`git reset` — Move HEAD Backward

```
# Soft reset — undo commit, keep changes staged
git reset --soft HEAD~1

# Mixed reset (default) — undo commit, keep changes unstaged
git reset HEAD~1
git reset --mixed HEAD~1

# Hard reset — undo commit AND discard all changes (destructive!)
git reset --hard HEAD~1

# Reset to a specific commit
git reset --hard a3f7c21
```

Reset modes explained

Mode	Commit	Staging Area	Working Directory
`--soft`	Undone	Unchanged (staged)	Unchanged
`--mixed`	Undone	Cleared	Unchanged
`--hard`	Undone	Cleared	**Discarded**

> Use `--soft` when you want to recommit with a better message.
> Use `--hard` only locally.

`git revert` — Safe Undo for Shared Branches

```
# Create a new commit that undoes the changes of a previous commit
git revert a3f7c21

# Revert the last commit
git revert HEAD

# Revert without immediately committing (stage only)
git revert --no-commit a3f7c21
```

Why `revert` is safe for shared branches

`git commit --amend` — Fix the Last Commit

```
# Fix the commit message of the last commit
git commit --amend -m "fix: correct typo in authentication header"

# Add a forgotten file to the last commit
git add forgotten-file.js
git commit --amend --no-edit
```

> `--amend` rewrites the last commit's SHA. Only use it **before** pushing to a shared remote.

Summary — Core Concepts

Command	What it does
`git status`	Show working directory and staging area status
`git add <file>`	Stage a file for the next commit
`git commit -m "..."`	Create a commit with staged changes
`git log --oneline`	Compact history view
`git diff`	Show unstaged changes
`git diff --staged`	Show staged changes